

TRƯỜNG: ĐH CNTP TP.HCM KHOA: CÔNG NGHỆ THÔNG TIN BỘ MÔN: KHDL&TTNT MH: TH TRÍ TUỆ NHÂN TẠO	BUỔI 1. GIỚI THIỆU PYTHON	
---	------------------------------	---

A. MỤC TIÊU

- Tạo một chương trình viết bằng ngôn ngữ Python.
- Sử dụng được các hàm xuất, nhập chuẩn; các kiểu dữ liệu; khai báo biến; các phép toán số học, logic.
- Áp dụng được các cấu trúc rẽ nhánh, vòng lặp vào các bài toán cụ thể.
- Xây dựng được các hàm chức năng và các lớp hàm.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

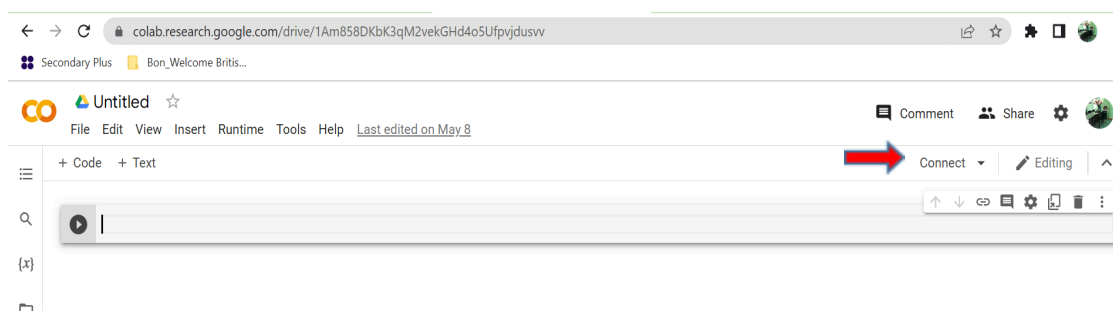
C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. Sử dụng Google Colab để viết chương trình:

B1. Truy cập địa chỉ: <https://colab.research.google.com/>

B2. Vào file  new notebook



B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. Nhập, xuất trong Python:

a. Nhập trong Python:

Cú pháp: `input("chuỗi ký tự: ")`

Ví dụ:



```
input("What is your name? ")
```

What is your name?

b. Xuất trong Python:

Cú pháp: `print(.....)`

Ví dụ:



```
name = input("What is your name? ")  
print("Hi " + name)
```

What is your name? John
Hi John

3. Biến:

a. Khái niệm: Biến là vùng chứa để lưu trữ các giá trị dữ liệu.

b. Tạo biến:

- Python không có lệnh để khai báo một biến.
- Một biến được tạo ngay khi bạn gán giá trị cho nó lần đầu tiên.
- Các biến không cần phải được khai báo với bất kỳ kiểu cụ thể nào và thậm chí có thể thay đổi kiểu sau khi chúng đã được thiết lập.
- Một biến có thể có tên ngắn (như `x` và `y`) hoặc tên mô tả hơn (age, carname, total, ...). Quy tắc cho các biến Python:
 - o Tên biến phải bắt đầu bằng một chữ cái hoặc ký tự gạch dưới
 - o Tên biến không thể bắt đầu bằng số
 - o Tên biến chỉ có thể chứa các ký tự chữ-số và dấu gạch dưới (A-z, 0-9 và `_`)
 - o Tên biến có phân biệt chữ hoa chữ thường (tuổi, Tuổi và TUỔI là ba biến khác nhau)

Ví dụ:

Khai báo biến	Kiểu dữ liệu	Ghi chú
<code>soluong = 10</code>	<code>type(soluong) == int</code>	#khai báo biến số lượng và gán trị là 10.

<code>dongia = 15.50</code>	<code>type(dongia) == float</code>	#khai báo biến đơn giá và gán trị là 15.50
<code>tensanpham = "lemon"</code> hoặc <code>tensanpham = 'lemon'</code>	<code>Type(tensanpham) == str</code>	#khai báo biến tên sản phẩm và gán trị là một chuỗi ký tự 'lemon'
<code>online = True</code>	<code>Type(bool) == bool</code>	#khai báo biến online và gán trị là True

c. Nhiều giá trị đến nhiều biến

Python cho phép bạn gán giá trị cho nhiều biến trong một dòng:

```
x, y, z = "Orange", "Banana", "Cherry"
print(x)
print(y)
print(z)
```

Hoặc bạn cũng có thể gán một giá trị cho nhiều biến :

```
x = y = z = "Orange"
print(x)
print(y)
print(z)
```

Nếu ta có một tập hợp giá trị trong một danh sách hoặc một tuple, ...Python cho phép trích xuất mỗi giá trị thành một biến.

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
print(z)
```

d. Biến toàn cục :

- Các biến được tạo bên ngoài một hàm (như trong tất cả các ví dụ ở trên) được gọi là các biến toàn cục.

- Các biến toàn cục có thể được sử dụng bởi tất cả mọi người, cả bên trong hàm và bên ngoài hàm.

Ví dụ :

```
 x = "awesome"

def myfunc():
    print("Python is " + x)

myfunc()
```

```
 Python is awesome
```

- Nếu bạn tạo một biến có cùng tên bên trong một hàm, thì biến này sẽ là cục bộ và chỉ có thể được sử dụng bên trong hàm. Biến toàn cục có cùng tên sẽ vẫn như cũ, toàn cục và với giá trị ban đầu.

Ví dụ :

```
 x = "awesome"

def myfunc():
    x = "fantastic"
    print("Python is " + x)

myfunc()

print("Python is " + x)
```

```
Python is fantastic
Python is awesome
```

- Thông thường, khi bạn tạo một biến bên trong một hàm, biến đó là cục bộ và chỉ có thể được sử dụng bên trong hàm đó.
- Để tạo một biến toàn cục bên trong một hàm, bạn có thể sử dụng từ khóa toàn cục.

Ví dụ :

```
def myfunc():
    global x
    x = "fantastic"

myfunc()

print("Python is " + x)
```

e. Ép kiểu :

Nếu bạn muốn chỉ định kiểu dữ liệu của một biến, điều này có thể được thực hiện với việc ép kiểu.

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

f. Lấy kiểu dữ liệu của biến :

Bạn có thể lấy kiểu dữ liệu của biến dựa vào hàm type(biến)

```
x = 5
y = 'John'
print(type(x))
print(type(y))

<class 'int'>
<class 'str'>
```

Các phép toán:

Phép toán	Mô tả
a = 5 b = 2	
a+b	7
a-b	3
a*b	10
a**b	25
a/b	2.5
a//b	2
<, <=, >, >=, ==, !=	Các phép toán so sánh

and, or, not	Các phép toán logic
bt1 and bt2	True khi và chỉ khi bt1 = bt2 = True
bt1 or bt2	False khi và chỉ khi bt1 = bt2 = False
not bt1	True khi bt1 = False và ngược lại

+, -, *, /, %

4. Kiểu dữ liệu:

Trong lập trình, kiểu dữ liệu là một khái niệm quan trọng. Các biến có thể lưu trữ dữ liệu thuộc nhiều loại khác nhau và các loại khác nhau có thể làm những việc khác nhau. Python có các kiểu dữ liệu sau được tích hợp sẵn theo mặc định, trong các danh mục sau:

Text Type: `str`

Numeric Types: `int`, `float`, `complex`

Sequence Types: `list`, `tuple`, `range`

Mapping Type: `dict`

Set Types: `set`, `frozenset`

Boolean Type: `bool`

Binary Types: `bytes`, `bytearray`, `memoryview`

None Type: `NoneType`

Trong Python, kiểu dữ liệu được đặt khi bạn gán giá trị cho một biến:

Example	Data Type
x = "Hello World"	str
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range
x = {"name" : "John", "age" : 36}	dict
x = {"apple", "banana", "cherry"}	set
x = frozenset({"apple", "banana", "cherry"})	frozenset
x = True	bool
x = b"Hello"	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview
x = None	NoneType

Xét ví dụ nhập vào năm sinh của bạn, tính tuổi và xuất ra màn hình.

```
birth_year = input("Enter your birth year: ")
age = 2022 - birth_year
print(age)
```

Enter your birth year: 1980

TypeError Traceback (most recent call last)

<ipython-input-15-585fa75571e4> in <module>

1 birth_year = input("Enter your birth year: ")

----> 2 age = 2022 - birth_year

3 print(age)

TypeError: unsupported operand type(s) for -: 'int' and 'str'

Hàm input luôn nhận giá trị chuỗi do đó để tính được tuổi ta phải chuyển giá trị của biến *birth_year* sang kiểu cùng kiểu *int* với năm 2022. Khi đó:

age = 2022 – int(birth_year)

Cú pháp: <kiểu cần chuyển> <tên biến hoặc hằng>

Câu lệnh	Ý nghĩa
int(<i>value</i>)	Chuyển <i>value</i> sang số nguyên.
float(<i>value</i>)	Chuyển <i>value</i> sang số thực.
bool(<i>value</i>)	Chuyển <i>value</i> sang boolean .
str(<i>value</i>)	Chuyển <i>value</i> sang chuỗi.

5. Câu lệnh rẽ nhánh và vòng lặp

a. If, elif và else

Lệnh if đơn không có else :

```
▶ x = int(input('Enter a value x = '))  
if x < 0:  
    print("It's negative")
```

Enter a value x = -10
It's negative

Lệnh if đơn có else :

```
▶ x = int(input('Enter a value x = '))  
if x < 0:  
    print("It's negative")  
else:  
    print("It's positive")
```

Enter a value x = 10
It's positive

Lệnh if lồng nhau:


```
x = int(input('Enter a value x = '))
if x < 0:
    print("It's negative")
elif x == 0:
    print("Equal to zero")
elif x <= 5:
    print("Positive but smaller than or equal to 5")
else:
    print("Positive and larger than 5")
```

Enter a value x = 10
Positive and larger than 5

Bài tập: viết chương trình giải phương trình bậc nhất, bậc 2.

b. Vòng lặp For, While:

- Cấu trúc chuẩn của vòng lặp **for**:

```
for value in collection:
    # do something with value
```

Ví dụ:

```
for i in range(5):
    print(i)
```

0
1
2
3
4

Tính tổng các giá trị khác None trong list: <pre>sequence = [1, 2, None, 4, None, 5] total = 0 for value in sequence: if value is None: continue total += value</pre>	Tính tổng các giá trị trong list, nếu giá trị là 5 thì thoát khỏi vòng lặp: <pre>sequence = [1, 2, 0, 4, 6, 5, 2, 1] total_until_5 = 0 for value in sequence: if value == 5: break total_until_5 += value</pre>
--	--

--	--

Viết chương trình in ra các cặp số (0,0), (1,0), (1,1), (2,0), (2,1), (2,2), (3,0), (3,1), (3,2), (3,3)

```
for i in range(4):
    for j in range(4):
        if j > i:
            break
        print((i, j))
```

– Cấu trúc chuẩn của vòng lặp **while**:

While condition:

Commant block

Ví dụ:

```
x = 256
total = 0
while x > 0:
    if total > 500:
        break
    total += x
    x = x // 2
```

6. **Range**: hàm range() trả về một tiến trình lặp nhằm tạo ra một chuỗi các số nguyên cách đều nhau.

Ví dụ:

```
In [122]: range(10)
Out[122]: range(0, 10)
```

```
In [123]: list(range(10))
Out[123]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [124]: list(range(0, 20, 2))
Out[124]: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

```
In [125]: list(range(5, 0, -1))
Out[125]: [5, 4, 3, 2, 1]
```

Viết chương trình tính tổng các số từ 1 đến 100.

```
sum = 0
for i in range(101):
    sum += i
print(sum)
```

5050

7. List:

<tên biến> = <[M1, M2, ..., Mn]> , Mi có thể cùng kiểu dữ liệu hoặc khác kiểu.

Ví dụ :

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
```

```
list1 = ["abc", 34, True, 40, "male"]
```

- Chiều dài list được xác định qua hàm len()

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

- Chúng ta cũng có thể sử dụng cấu trúc list() để tạo ra một list mới.

```
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

- Truy xuất đến các item của list

```
[50] thislist = list(("banana", "apple", 'mango'))
print(thislist[1])
print(thislist[-1])
print(thislist[:2])
print(thislist[-3:-1])
```

```
apple
mango
['banana', 'apple']
['banana', 'apple']
```

- Lặp thông qua list

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
    print(x)
```

- Lặp thông qua các số chỉ mục

```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
    print(thislist[i])
```

Các phương thức của list.

Method	Description
<u>append()</u>	Adds an element at the end of the list
<u>clear()</u>	Removes all the elements from the list
<u>copy()</u>	Returns a copy of the list
<u>count()</u>	Returns the number of elements with the specified value
<u>extend()</u>	Add the elements of a list (or any iterable), to the end of the current list
<u>index()</u>	Returns the index of the first element with the specified value
<u>insert()</u>	Adds an element at the specified position
<u>pop()</u>	Removes the element at the specified position
<u>remove()</u>	Removes the item with the specified value
<u>reverse()</u>	Reverses the order of the list
<u>sort()</u>	Sorts the list

8. Dictionary:

- Tạo dict:

```
dict1 = {
    "brand": "Ford",
    "electric": False,
    "year": 1964,
    "colors": ["red", "white", "blue"]
}
```

- Thêm, sửa, xuất một item.

```
print(dict1)
dict1["colors"].append("yellow")
print(dict1)
dict1["electric"] = True
print(dict1)
dict1["month"] = 10
print(dict1)
print(dict1["brand"], dict1["year"])
```

```
{'brand': 'Ford', 'electric': False, 'year': 1964, 'colors': ['red', 'white', 'blue']}
{'brand': 'Ford', 'electric': False, 'year': 1964, 'colors': ['red', 'white', 'blue', 'yellow']}
{'brand': 'Ford', 'electric': True, 'year': 1964, 'colors': ['red', 'white', 'blue', 'yellow']}
{'brand': 'Ford', 'electric': True, 'year': 1964, 'colors': ['red', 'white', 'blue', 'yellow'], 'month': 10}
Ford 1964
```

- Xóa một item hoặc dict

```
dict1 = {
    "brand": "Ford",
    "electric": False,
    "year": 1964,
    "colors": ["red", "white", "blue"]
}
dict1.pop("electric")
print(dict1)
dict1.popitem()
print(dict1)
del dict1["year"]
print(dict1)
dict1.clear()
print(dict1)
```

```
{'brand': 'Ford', 'year': 1964, 'colors': ['red', 'white', 'blue']}
{'brand': 'Ford', 'year': 1964}
{'brand': 'Ford'}
{}
```

- Lặp trong dictionary

```
for k in dict1:
    print(k)
```

```
brand
electric
year
colors
```

```
for k in dict1.keys():
    print(k)
```

```
brand
electric
year
colors
```



```
for k in dict1:  
    print(dict1[k])
```

```
Ford  
False  
1964  
['red', 'white', 'blue']
```



```
for k in dict1.values():  
    print(k)
```

```
Ford  
False  
1964  
['red', 'white', 'blue']
```

```
for k,v in dict1.items():  
    print(k, v)
```

```
brand Ford  
electric False  
year 1964  
colors ['red', 'white', 'blue']
```

- Các phương thức của Dictionary

Method	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and value
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>values()</code>	Returns a list of all the values in the dictionary

9. Function:

- Tạo hàm, gọi hàm

```
def my_function():
    print("Hello from a function")

my_function()
```

- Đối số của hàm

```
def my_function(fname, lname):
    print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

```
def my_function(food):
    for x in food:
        print(x)
fruits = ["apple", "banana", "cherry"]
my_function(fruits)
```

```
apple
banana
cherry
```

- Hàm đệ quy

```
def tri_recursion(k):
    if(k > 0):
        result = k + tri_recursion(k - 1)
        print(result)
    else:
        result = 0
    return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

10. Lớp/Đối tượng

Python là ngôn ngữ lập trình hướng đối tượng và hầu hết mọi thứ trong Python đều là một đối tượng, với các thuộc tính và phương thức của nó.

Một Lớp giống như một phương thức khởi tạo đối tượng, hoặc một "bản thiết kế-blueprint" để tạo các đối tượng.

- Tạo lớp: để tạo lớp ta sử dụng từ khóa class

```
class MyClass:
    x = 5
```

- Tạo đối tượng: ta sử dụng tên lớp để tạo đối tượng.

```
p1 = MyClass()
print(p1.x)
```

- Hàm `__init__()`: Để hiểu ý nghĩa của các lớp, chúng ta phải hiểu hàm `__init__()` có sẵn. Tất cả các lớp đều có một hàm được gọi là `__init__()`, hàm này luôn

được thực thi khi lớp đang được khởi tạo. Chúng ta có thể sử dụng hàm `__init__` () để gán giá trị cho thuộc tính của đối tượng hoặc các thao tác khác thực sự cần thực hiện khi đối tượng đang được tạo.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

print(p1.name)
print(p1.age)
```

- Hàm `__str__` (): hàm `__str__` () kiểm soát những gì sẽ được trả về khi đối tượng lớp được biểu diễn dưới dạng một chuỗi. Ví dụ trên là định nghĩa lớp mà không có hàm `__str__` (). Biểu diễn chuỗi của một đối tượng VỚI hàm `__str__` ():

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}({self.age})"

p1 = Person("John", 36)

print(p1)
```

- Các phương thức của đối tượng: các đối tượng cũng có thể chứa các phương thức và đó chính là các hàm thuộc về đối tượng. Ví dụ chúng tôi tạo một phương thức trong lớp `Person`:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

- Tham số self: là một tham chiếu đến thể hiện hiện tại của lớp và được sử dụng để truy cập các biến thuộc về lớp.

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc):
        print("Hello my name is " + abc.name)

p1 = Person("John", 36)
p1.myfunc()
```

- Thay đổi thuộc tính đối tượng:

```
p1.age = 40
```

- Xóa thuộc tính/đối tượng:

```
del p1.age
```

```
del p1
```

11. NumPy Basics: Arrays and Vectorized Computation

- Mảng đa chiều: một trong những tính năng chính của NumPy là đối tượng mảng N-chiều, hay ndarray, là một vùng chứa linh hoạt, nhanh chóng cho các tập dữ

liệu lớn trong Python. Mảng cho phép bạn thực hiện các phép toán trên toàn bộ khối dữ liệu bằng cách sử dụng cú pháp tương tự với các phép toán tương đương giữa các phần tử vô hướng.

✓
0s

```
[6] import numpy as np  
     data = np.random.randn(2,3)  
     data
```

```
array([[ -0.33167072, -1.2294598 , -0.32197215],  
       [-2.54324423, -0.94924727,  1.7721657 ]])
```

✓
0s

```
[7] data = data*100  
     data
```

```
array([[ -33.16707182, -122.94598041,  -32.19721473],  
       [-254.32442256,  -94.9247269 ,  177.21657027]])
```

✓
0s



```
data = data + data  
data
```

```
array([[ -66.33414365, -245.89196082,  -64.39442946],  
       [-508.64884512, -189.8494538 ,  354.43314054]])
```

```
✓ [10] import numpy as np
0s      data = np.random.randn(2,3)
      data

array([[ -0.45376706, -0.07264972, -0.92627394],
       [-0.02919274,  1.83597085, -1.22126133]])
```

```
✓ [11] data = data/10
0s      data

array([[ -0.04537671, -0.00726497, -0.09262739],
       [-0.00291927,  0.18359708, -0.12212613]])
```

```
✓ [12] data = data - data
0s      data

array([[0., 0., 0.],
       [0., 0., 0.]])
```

Để biết được kích thước và kiểu dữ liệu của mảng ta thực hiện như sau:

```
In [17]: data.shape
Out[17]: (2, 3)

In [18]: data.dtype
Out[18]: dtype('float64')
```

b. Tạo ndarrays

Cách dễ nhất để tạo một mảng là sử dụng hàm mảng.

```
In [19]: data1 = [6, 7.5, 8, 0, 1]

In [20]: arr1 = np.array(data1)

In [21]: arr1
Out[21]: array([ 6. ,  7.5,  8. ,  0. ,  1. ])
```

```
In [22]: data2 = [[1, 2, 3, 4], [5, 6, 7, 8]]
```

```
In [23]: arr2 = np.array(data2)
```

```
In [24]: arr2
```

```
Out[24]:
```

```
array([[1, 2, 3, 4],  
       [5, 6, 7, 8]])
```

```
In [25]: arr2.ndim
```

```
Out[25]: 2
```

```
In [26]: arr2.shape
```

```
Out[26]: (2, 4)
```

Ngoài `np.array`, có một số chức năng khác để tạo mảng mới. Ví dụ, số không và số không tạo ra các mảng 0 hoặc 1, tương ứng với độ dài hoặc hình dạng nhất định hay tạo ra một mảng rỗng. Để tạo mảng có chiều cao hơn với các phương thức này, hãy truyền một bộ giá trị cho hình dạng:

```
In [29]: np.zeros(10)
```

```
Out[29]: array([ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.])
```

```
In [30]: np.zeros((3, 6))
```

```
Out[30]:
```

```
array([[ 0.,  0.,  0.,  0.,  0.,  0.],  
       [ 0.,  0.,  0.,  0.,  0.,  0.],  
       [ 0.,  0.,  0.,  0.,  0.,  0.]])
```



```
np.empty((2, 3, 2))
```

```
array([[[[1.14867970e-316, 2.14321575e-312],  
         [2.37663529e-312, 2.56761491e-312],  
         [8.48798317e-313, 9.33678148e-313]],  
       [[1.08221785e-312, 6.79038653e-313],  
         [8.70018275e-313, 2.02566915e-322],  
         [2.13623551e-316, 0.00000000e+000]]]])
```

```
In [32]: np.arange(15)
```

```
Out[32]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14])
```

c. Các kiểu dữ liệu của ndarray

Kiểu dữ liệu hoặc dtype là một đối tượng đặc biệt chứa thông tin (hoặc siêu dữ liệu, dữ liệu về dữ liệu) mà ndarray cần diễn giải một đoạn bộ nhớ là một kiểu dữ liệu cụ thể:

```
In [33]: arr1 = np.array([1, 2, 3], dtype=np.float64)
```

```
In [34]: arr2 = np.array([1, 2, 3], dtype=np.int32)
```

Bạn có thể chuyển đổi hoặc ép kiểu dữ liệu của mảng từ kiểu này sang kiểu khác bằng cách sử dụng phương thức `astype` của ndarray:

```
In [37]: arr = np.array([1, 2, 3, 4, 5])
```

```
In [38]: arr.dtype
```

```
Out[38]: dtype('int64')
```

```
In [39]: float_arr = arr.astype(np.float64)
```

```
In [40]: float_arr.dtype
```

```
Out[40]: dtype('float64')
```

Trong ví dụ này, các số nguyên được chuyển sang số thực. Nếu chuyển từ số thực sang số nguyên thì phần thập phân sẽ bị cắt bớt:

```
In [41]: arr = np.array([3.7, -1.2, -2.6, 0.5, 12.9, 10.1])
```

```
In [42]: arr
```

```
Out[42]: array([ 3.7, -1.2, -2.6,  0.5, 12.9, 10.1])
```

```
In [43]: arr.astype(np.int32)
```

```
Out[43]: array([ 3, -1, -2,  0, 12, 10], dtype=int32)
```

Nếu ta có một mảng chuỗi số, ta có thể sử dụng phương thức `astype` để chuyển thành số.

```
In [44]: numeric_strings = np.array(['1.25', '-9.6', '42'], dtype=np.string_)
```

```
In [45]: numeric_strings.astype(float)
```

```
Out[45]: array([ 1.25, -9.6 , 42.  ])
```

d. Số học với mảng Numpy.

Mảng rất quan trọng vì chúng cho phép bạn thể hiện các hoạt động hàng loạt trên dữ liệu mà không cần viết bất kỳ vòng lặp for nào. Người dùng NumPy gọi đây là vector hóa. Bất kỳ phép toán số học nào giữa các mảng có kích thước bằng nhau đều có thể áp dụng.

```
In [51]: arr = np.array([[1., 2., 3.], [4., 5., 6.]])
```

```
In [52]: arr
```

```
Out[52]:
```

```
array([[ 1.,  2.,  3.],  
       [ 4.,  5.,  6.]])
```

```
In [53]: arr * arr
```

```
Out[53]:
```

```
array([[ 1.,  4.,  9.],  
       [16., 25., 36.]])
```

```
In [54]: arr - arr
```

```
Out[54]:
```

```
array([[ 0.,  0.,  0.],  
       [ 0.,  0.,  0.]])
```

```
In [55]: 1 / arr
```

```
Out[55]:
```

```
array([[ 1.    ,  0.5   ,  0.3333],  
       [ 0.25  ,  0.2   ,  0.1667]])
```

```
In [56]: arr ** 0.5
```

```
Out[56]:
```

```
array([[ 1.    ,  1.4142,  1.7321],  
       [ 2.    ,  2.2361,  2.4495]])
```

e. Indexing with slices

```
[4] import numpy as np
    arr1 = np.arange(10)
    arr1
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[5] arr1[2:6] = 10
    arr1
```

```
array([ 0,  1, 10, 10, 10, 10,  6,  7,  8,  9])
```

```
[6] arr1[:5]
```

```
array([ 0,  1, 10, 10, 10])
```

```
 arr1[-5:]
```

```
array([10,  6,  7,  8,  9])
```

```
[10] arr2d = np.array([[1,2,3], [4,5,6], [7,8,9]])
    arr2d
```

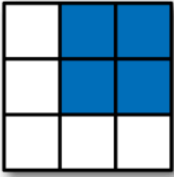
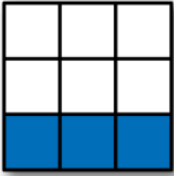
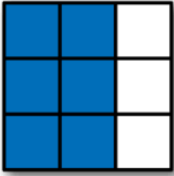
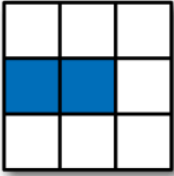
```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

```
[14] arr2d[:2, :2]
```

```
array([[1, 2],
       [4, 5]])
```

```
 arr2d[:, :1]
```

```
array([[1],
       [4],
       [7]])
```


	Expression	Shape
	<code>arr[:2, 1:]</code>	<code>(2, 2)</code>
	<code>arr[2]</code> <code>arr[2, :]</code> <code>arr[2:, :]</code>	<code>(3,)</code> <code>(3,)</code> <code>(1, 3)</code>
	<code>arr[:, :2]</code>	<code>(3, 2)</code>
	<code>arr[1, :2]</code> <code>arr[1:2, :2]</code>	<code>(2,)</code> <code>(1, 2)</code>

II. Bài tập hướng dẫn mẫu

- Viết chương trình nhập vào và xuất ra tên, tuổi của một khách hàng.

```
[2] name = input("Your name is: ")
    age = input("Age : ")
    print(f"\nMy name is {name}")
    print("\nAge = " + age)
```

Your name is: Lan

Age : 42

My name is Lan

Age = 42

- Viết chương trình tính tiền thuê Taxi. Biết rằng, mỗi khách hàng tính 2£ và 1.5£ cho 1km.

```
[3] def taxi():
    passengers = (int)(input("\nNhap vao so nguoi: "))
    distance = (float)(input("\nhap vao so km: "))
    total = 2*passengers + 1.5* distance
    print("\nTotal: ", total)
```

 taxi()

Nhap vao so nguoi: 4

hap vao so km: 3

Total: 12.5

3. Viết chương trình nhập vào một chuỗi ký tự. Xuất tất cả các ký tự ở vị trí chẵn.

```
[1] string = input("\nstring enter: ")
    print(string[::2])
```

string enter: abcdefg
aceg

4. Viết chương trình tính tổng các số từ 0 đến n mà là bội của 3 hoặc 5.

```
 sum = 0
n = 11
for i in range(n):
    if i%3 == 0 or i%5 == 0:
        sum += i
print(f"\nTong cac so tu 0 den {n} ma la boi 3 hoac 5 la: {sum}")
```

Tong cac so tu 0 den 11 ma la boi 3 hoac 5 la: 33

5. Viết hàm trộn 2 mảng một chiều thành 1 mảng một chiều với mỗi phần tử của mảng mới là tổng của 2 phần tương ứng từ 2 mảng cho trước. Trong quá trình trộn 2 mảng nếu mảng nào còn phần tử thì các phần tử còn lại của mảng đó sẽ đưa vào mảng mới.

Ví dụ:

- Mảng a: 3 9 1 4
- Mảng b: 2 7 4 3 2 8
- Mảng kết quả: 5 16 5 7 2 8 23.

```

a = [3, 9, 1, 4]
b = [2, 7, 4, 3, 8, 2]
min = len(b) if len(a)>len(b) else len(a)
c = a if len(a)>len(b) else b
for i in range(min):
    c[i] = a[i] + b[i]
print(c)

```

```

[5, 16, 5, 7, 8, 2]

```

6. Tạo một list trái cây (bằng tiếng anh) fruits. Từ list fruits tạo một list mới chỉ chứa các loại trái cây có chứa ký tự “a”.

```

[53] fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []
for x in fruits:
    if "a" in x:
        newlist.append(x)
print(newlist)

```

```

['apple', 'banana', 'mango']

```

```

fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if "a" in x]
print(newlist)

```

```

['apple', 'banana', 'mango']

```

7. Tạo một array chứa các số nguyên, sau đó tạo một filter array chứa các giá trị lớn hơn 42.

```

import numpy as np

arr = np.array([41, 42, 43, 44])

# Create an empty list
filter_arr = []

# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the value to True, otherwise False:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)

```

8. Tạo một mảng filter chỉ chứa các phần tử chẵn từ mảng gốc.

```

import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

filter_arr = arr % 2 == 0

newarr = arr[filter_arr]

print(filter_arr)
print(newarr)

```

9. Một cuốn sách gồm các thông tin sau:

Mã sách là chuỗi có 10 ký tự

Tên sách là chuỗi tối đa 20 ký tự

Giá bán là một số thực

Số lượng là một số nguyên a.

- a. Xây dựng cấu trúc SACH mô tả cuốn sách
- b. Cho một mảng có n cuốn sách.

```

class Sach:
    dic_sach = {"masach": [],
                "tensach": [],
                "dongia": [],
                "soluong": []
                }
    def __init__(self, n):
        for i in range(n):
            ma = input("Nhap ma sach: ")
            ten = input("Nhap ten sach: ")
            dg = float(input("Nhap don gia: "))
            sl = int(input("Nhap so luong: "))
            self.dic_sach['masach'].append(ma)
            self.dic_sach['tensach'].append(ten)
            self.dic_sach['dongia'].append(dg)
            self.dic_sach['soluong'].append(sl)

Sach(3)
print(Sach.dic_sach)

```

```

> Nhap ma sach: 01
  Nhap ten sach: Toan
  Nhap don gia: 25000
  Nhap so luong: 30
  Nhap ma sach: 02
  Nhap ten sach: Ly
  Nhap don gia: 20000
  Nhap so luong: 30
  Nhap ma sach: 03
  Nhap ten sach: Hoa
  Nhap don gia: 20000
  Nhap so luong: 10
{'masach': ['01', '02', '03'], 'tensach': ['Toan', 'Ly', 'Hoa'], 'dongia': [25000.0, 20000.0, 20000.0], 'soluong': [30, 30, 10]}

```

c. Viết hàm in ra màn hình tổng số lượng các cuốn sách.

```

[15] print("Tong so luong sach: " + str(sum(Sach.dic_sach["soluong"])))

Tong so luong sach: 70

```

d. Viết hàm xuất các sách có số lượng > 10.

```

for i in range(len(Sach.dic_sach["masach"])):
    if Sach.dic_sach["soluong"][i] > 10:
        print(Sach.dic_sach["tensach"][i])

```

III. Bài tập ở lớp

- Viết chương trình nhập vào 2 giá trị a, b và tính tổng, hiệu, tích thương của a và b.

2. Lấy các ký tự từ chỉ số 2 đến chỉ số 4 (llo) của chuỗi “Hello World”.
3. Xóa khoảng trắng thừa ở đầu và cuối chuỗi “ Hello World ”. Chuyển chuỗi sang dạng hoa và thường. Thay ký tự H thành ký tự J.
4. In ra “Hello World!” nếu a lớn hơn b.
5. In ra “Yes” nếu a bằng b, ngược lại in “No”.
6. In 1 nếu a bằng b, in 2 nếu $a > b$, in 3 nếu ngược lại.
7. In ra “Hello World!” nếu a bằng b và b bằng d.
8. In ra “Hello World!” nếu a bằng b hoặc c bằng d.
9. Viết lại câu lệnh if ... else sau lên cùng 1 dòng.

```
if a > b:
    print('Yes')
else:
    print('No')
```

10. Nhập vào 2 số a, b. Thực hiện câu lệnh và cho biết kết quả

```
print("A") if a > b else print("=") if a == b else print("B")
```

11. Tạo mảng số nguyên a với n phần tử và mảng b chỉ chứa các phần tử chẵn của a.
12. Viết chương trình tính tổng các số từ 0 đến 999 mà là bội của 3 hoặc 5.
13. Viết hàm trộn 2 mảng một chiều thành 1 mảng một chiều với mỗi phần tử của mảng mới là tổng của 2 phần tương ứng từ 2 mảng cho trước. Trong quá trình trộn 2 mảng nếu mảng nào còn phần tử thì các phần tử còn lại của mảng đó sẽ đưa vào mảng mới.

Ví dụ:

Mảng a: 3 9 1 4

Mảng b: 2 7 4 3 2 8

Mảng kết quả: 5 16 5 7 2 8 23.

14. Cho mảng 2 chiều a có m dòng, n cột chứa số nguyên, viết các hàm sau:

- a. Tạo mảng a chứa các số nguyên ngẫu nhiên.
- b. Xuất các phần tử thuộc dòng k.
- c. Xuất các phần tử thuộc cột k.
- d. Tìm dòng có tổng lớn nhất nhất 45.
- e. Tìm cột có tích nhỏ nhất.
- f. Xuất ra các phần tử thuộc dòng chẵn và cột lẻ trong a.
- g. Tính trung bình cộng các phần tử chẵn thuộc dòng lẻ của a.

- h. Tính trung bình cộng các phần tử thuộc biên.
- i. Tính trung bình tích các phần tử không thuộc biên.

15. Một sinh viên gồm các thông tin sau:

Mã sinh viên là chuỗi có 10 ký tự

Tên là chuỗi tối đa 20 ký tự

Năm sinh là một số nguyên

Điểm trung bình là một số thực a.

- a. Xây dựng cấu trúc SINHVIEN mô tả một sinh viên
- b. Cho một mảng có n sinh viên.
- c. Viết hàm cho biết có bao sinh viên đủ điều kiện lên lớp, biết rằng sinh viên đủ điều kiện lên lớp khi điểm trung bình lớn hơn hoặc bằng 5.
- d. Xuất các sinh viên đủ 20 tuổi.
- e. Đếm số sinh viên học hệ đại học, biết rằng sinh viên hệ DH có mã sinh viên chứa 2 ký tự DH ở vị trí 2,3 trong chuỗi. VD: 02DH0001.
- f. Cho biết trong mảng có bao nhiêu sinh viên có tên «Lan»
- g. Cho biết trong mảng có bao nhiêu sinh viên có họ «Phan»

IV. Bài tập về nhà

1. Viết hàm trộn 2 mảng một chiều thành 1 mảng một chiều với mỗi phần tử của mảng mới là min của 2 phần tương ứng từ 2 mảng cho trước. Trong quá trình trộn 2 mảng nếu mảng nào còn phần tử thì các phần tử còn lại của mảng đó sẽ đưa vào mảng mới.

Ví dụ:

Mảng a: 3 9 1 4

Mảng b: 2 7 4 3 2 8

Mảng kết quả: 2 7 1 3 2 8

2. Cho mảng 2 chiều a có m dòng, n cột chứa số nguyên, viết các hàm sau:
 - a. Tính tổng các phần tử thuộc tam giác trên của đường chéo phụ kể cả đường chéo phụ trong ma trận vuông a cấp n.
 - b. Chuyển các phần tử âm thành trị tuyệt đối của nó trong a.
 - c. Thay các phần tử chẵn trong a bằng số nguyên x cho trước.
 - d. Kiểm tra a có toàn chẵn không?
 - e. Cho ma trận vuông a cấp n, viết hàm kiểm tra a có đối xứng không. Biết rằng

ma trận đối xứng là ma trận có $a[i][j] = a[j][i]$.

f. Kiểm tra ma trận vuông a cấp n có đường chéo chính tăng dần không?

g. Xuất các phần tử thuộc tam giác dưới của đường chéo phụ kể cả đường chéo phụ trong ma trận vuông a cấp n.

h. Kiểm tra ma trận vuông a cấp n có đường chéo phụ giảm dần không?

3. Tạo một dictionary chứa 3 dictionaries.
4. Đọc và xuất file trong Python.
5. Tìm hiểu Pandas và Matplotlib trong Python